

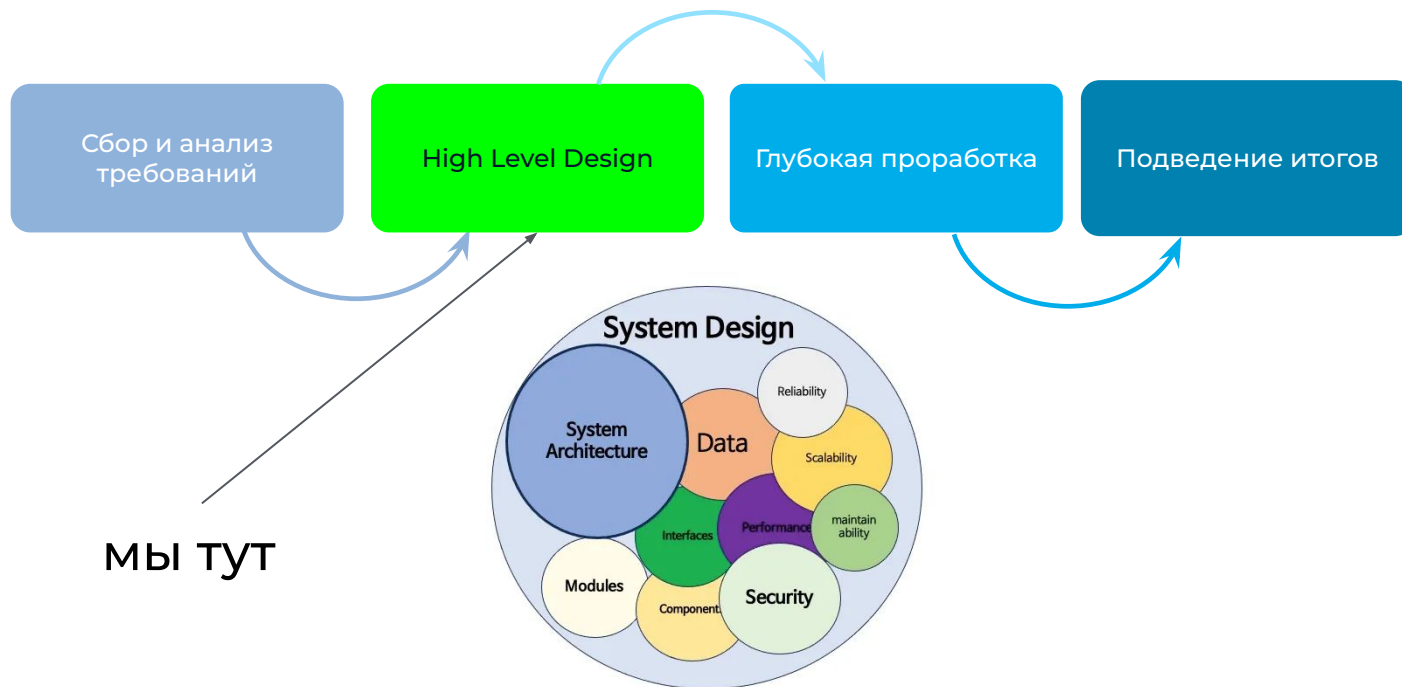


НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №4
Архитектура. Часть №2

Этапность





Модульность и связанность

Как выбрать границы?

- По бизнес-домену
- По распоряжению тимлида
- По гадальному шару
- По особым практикам

Cohesion

Связь элементов внутри модуля

high cohesion: модуль сфокусирован на решении одной узкой проблемы, а не занимается выполнением разнородных функций

Coupling

Зависимость между модулями

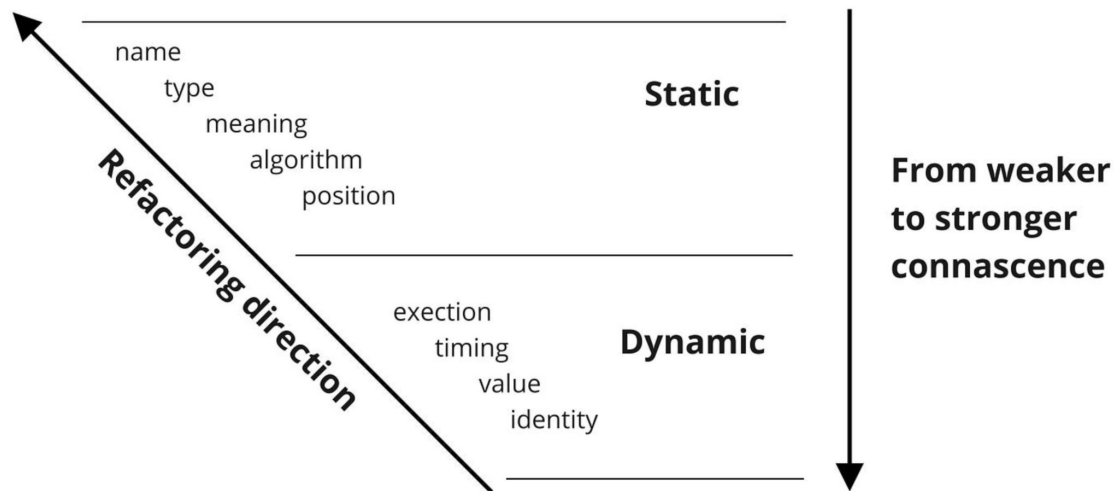
low/loose coupling: модули должны иметь возможность взаимодействовать, но при этом как можно меньше знать друг о друге



Connascence

Две сущности connascent, если изменение одной требует изменения другой.

Насколько больно менять?
Насколько сильное влияние оказывают?





Connascence of Name

Было

```
# users_service.py
def get_user(user_id: int) -> User:

# orders_service.py
user = get_user(123) # знает имя "get_user"
```

Стало

```
# users_service.py
def fetch_user(user_id: int) -> User:

# orders_service.py
user = get_user(123) # ошибка
```

Было

```
REST API
GET /api/user/123
```

Стало

```
REST API
GET /api/users/123
```

Вывод

В рамках одного модуля быстро читится
Проблемы возникают, когда пересекает границу



Connascence of Type

Было

```
# users_service.py
def get_user(user_id: int) -> User:

# orders_service.py
user = get_user(123)
```

Стало

```
# users_service.py
def get_user(user_id: float) -> User:

# orders_service.py
user = get_user(123) # ошибка
```

Вывод

Тип это уже семантика, может оказывать более сильное влияние в рантайме



Connascence of Meaning

Магические числа

```
# order_service.py
```

```
if order.status == 2:  
    notify_courier()
```

```
# payment_service.py
```

```
if order.status == 2:  
    charge_customer()
```

```
# analytics_service.py
```

```
if order.status == 2:  
    increment_counter("confirmed_orders")
```

Что такое 2?

Договоренность о формате

1. Сервис А пишет в очередь:

```
{"ts": "2024-01-15T10:30:00"}
```

2. Сервис В читает и парсит:

```
datetime.strptime(msg["ts"], "%Y-%m-%dT%H:%M:%S")
```

Сервис А добавляет миллисекунды, сервис В падает

HTTP-коды

1. Один сервис:

```
200 OK + {"error": "user not found"} (???????????????)
```

2. Другой сервис:

```
404 Not Found (?????????????????????????????????????)
```

Вывод

Магические числа -> именованные константы или enums (переходим к CoN)

Форматы -> явные схемы (JSON Schema, Protobuf)

HTTP-договоренности -> задокументировать и следовать стандарту



Connascence of Position

Запускаем ракеты

```
# nukes.py

def get_user_details():
    # Returns a user's details as a list:
    # first_name, last_name, is_good_person, is_admin
    return ["Thomas", "Richards", True, False]

def launch_nukes(user):
    if user[3]:
        # actually launch the nukes
    else:
        raise PermissionDeniedError("User is not an administrator!")
```

Вывод

Можем снизить до CoN

Что делать: именованные параметры, объекты вместо позиционных списков, схемы с именами полей



Connascence of Algorithm

Меняем алгоритм

```
# auth_service.py — генерирует токен
def generate_token(user_id: str) -> str:
    return hmac.new(SECRET, f"{user_id}:v1".encode()).hexdigest()

# gateway.py — валидирует токен
def validate_token(user_id: str, token: str) -> bool:
    expected = hmac.new(SECRET, f"{user_id}:v1".encode()).hexdigest()
    return token == expected
```

Если меняем алгоритм в одном месте, то и в другом

Вывод

Что делать: общая библиотека, или перенести алгоритм в один сервис (другие просто вызывают его)



Connascence of Execution (dynamic)

Спецификация

Event: OrderCreated -> inventory-service
резервирует товар

Event: OrderPaid -> inventory-service кладет
товар в корзину

Event: OrderCancelled -> inventory-service
освобождает товар

Стало

Из-за сетевой задержки порядок изменился:

OrderCancelled

OrderCreated

OrderPaid

По итогу, товар остался в корзине навсегда

Вывод

Используем специальные механизмы, следящие за порядком выполнения:

1. State Machine
2. Идемпотентные операции
3. Saga pattern

Ловится на этапе тестирования (QA: “а что если изменить порядок?”)



Connascence of Timing (dynamic)

Race Condition

Два экземпляра сервиса обрабатывают заказ
одновременно

Instance A

order = db.get_order(id=42)

status = "new"

order.status = "processing"

db.save(order)

Instance B

order = db.get_order(id=42)

status = "new"

order.status = "processing"

db.save(order)

Заказ обработан дважды - не было блокировок

Инваридация кэша

product-service обновил цену товара

db.update("products", id=1, price=999)

кэш НЕ инвалидирован сразу

order-service читает цену

price = cache.get("product:1:price")

**Получает старую цену 1299 - кэш ещё живой, заказ
создан по старой цене**

Вывод

Используем блокировки, механизмы синхронизации времени и избегания clock skew.
Ловится в проде на больших объемах нагрузки



Connascence of Value (dynamic)

Timeouts

```
# frontend config
api_timeout: 1000ms
# backend service config
db_query_timeout: 3000ms
```

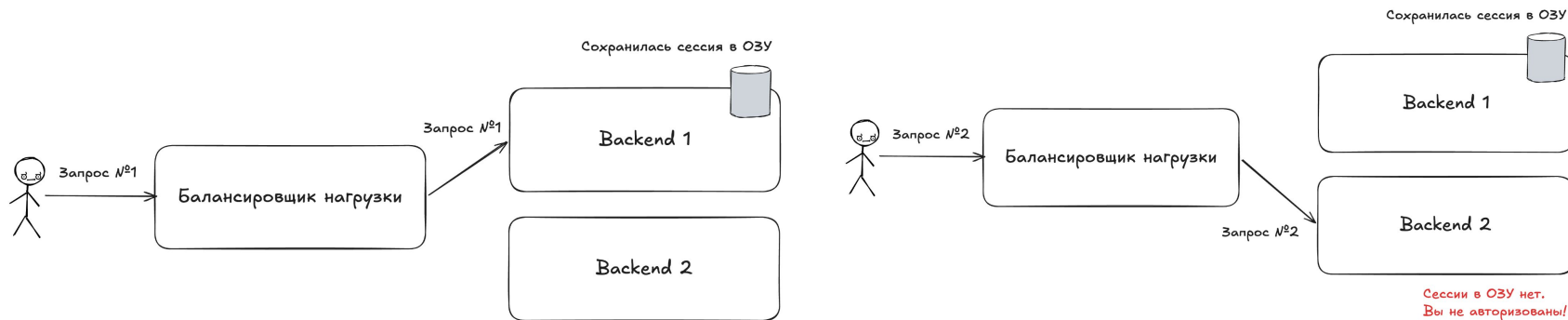
Пул соединений

```
# application config
db_pool_size: 100 # на каждый инстанс
# PostgreSQL config
max_connections: 200
```

Вывод

Используем явные зависимости между конфигурациями (по возможности, максимальная автоматизация), валидация конфигураций при старте, конфигурации - единый источник правды. Ловится чаще всего на поздних этапах.

Connascence of Identity (dynamic)

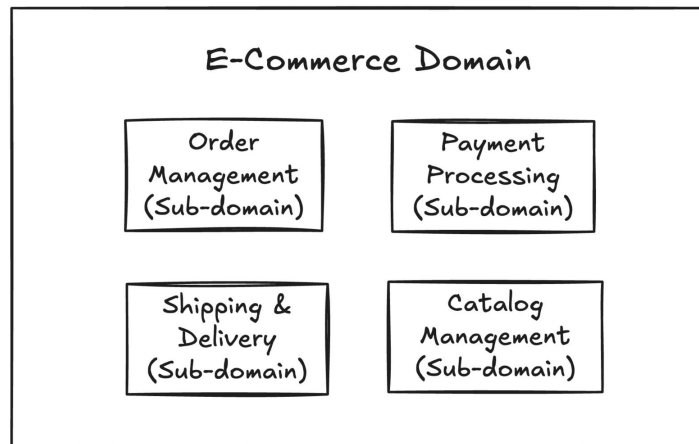
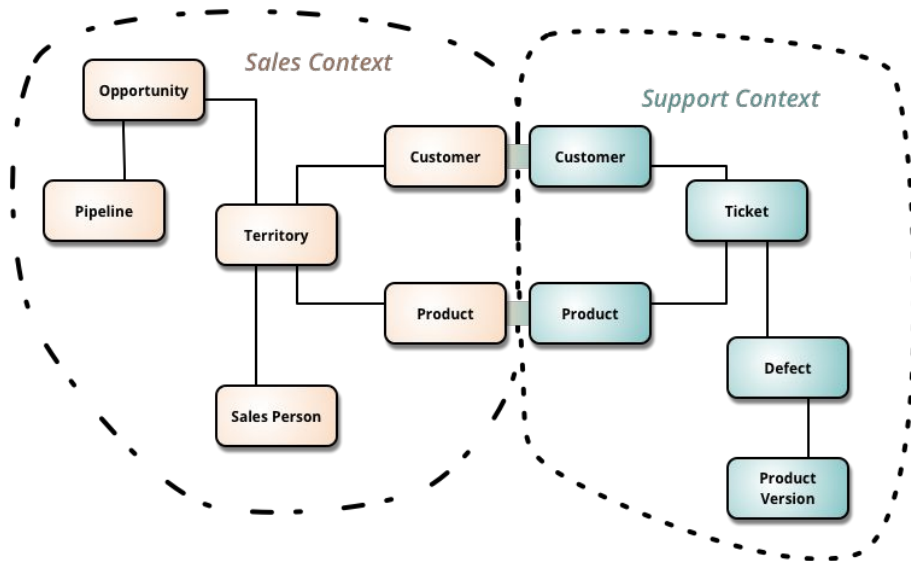


Вывод

Используем централизованное состояние (например, механизмы из серии sticky session), Saga и компенсирующие транзакции.
Сложно воспроизвести.

Причина появления

Код не отражает бизнес-логику. Границы модулей размыты, нарезаны по техническому удобству
Одни и те же слова означают разные вещи в разных частях системы
Фокус на Decompose by Subdomain (концентрируемся на бизнесе, не на технике)

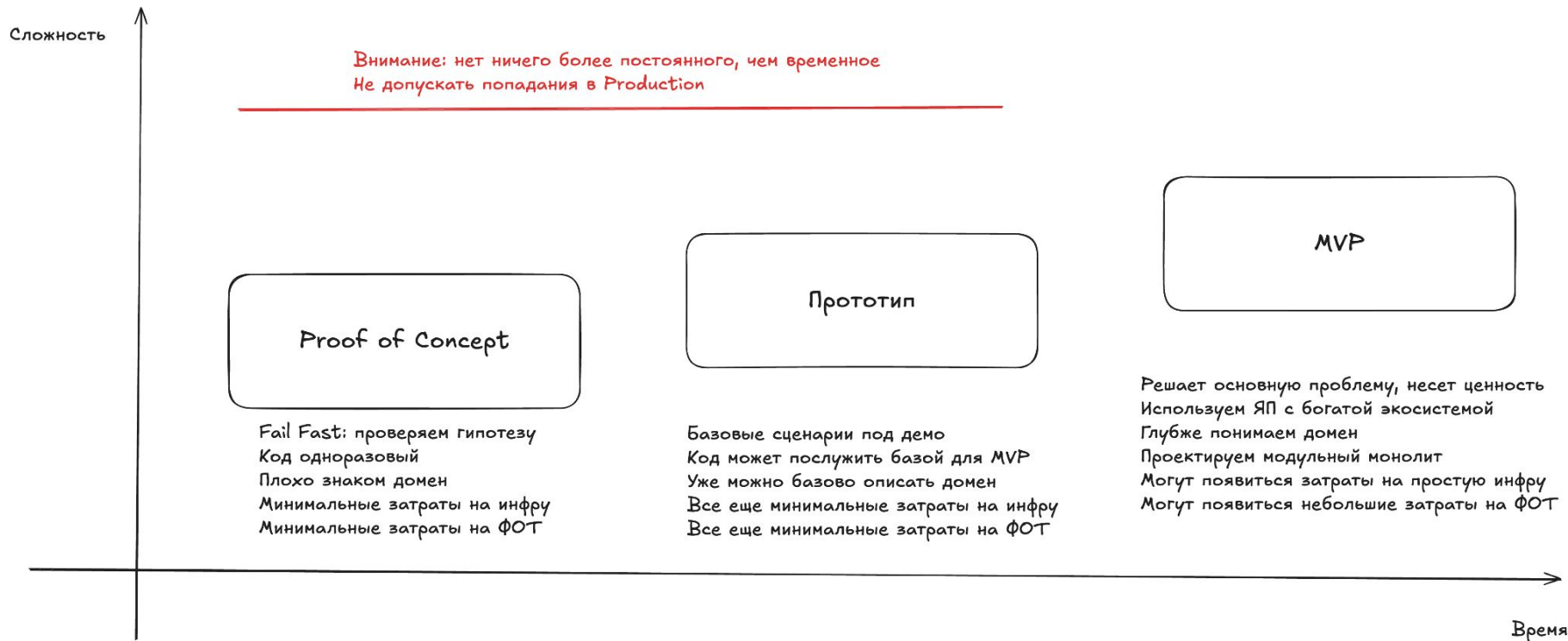


<https://www.eventstorming.com/>

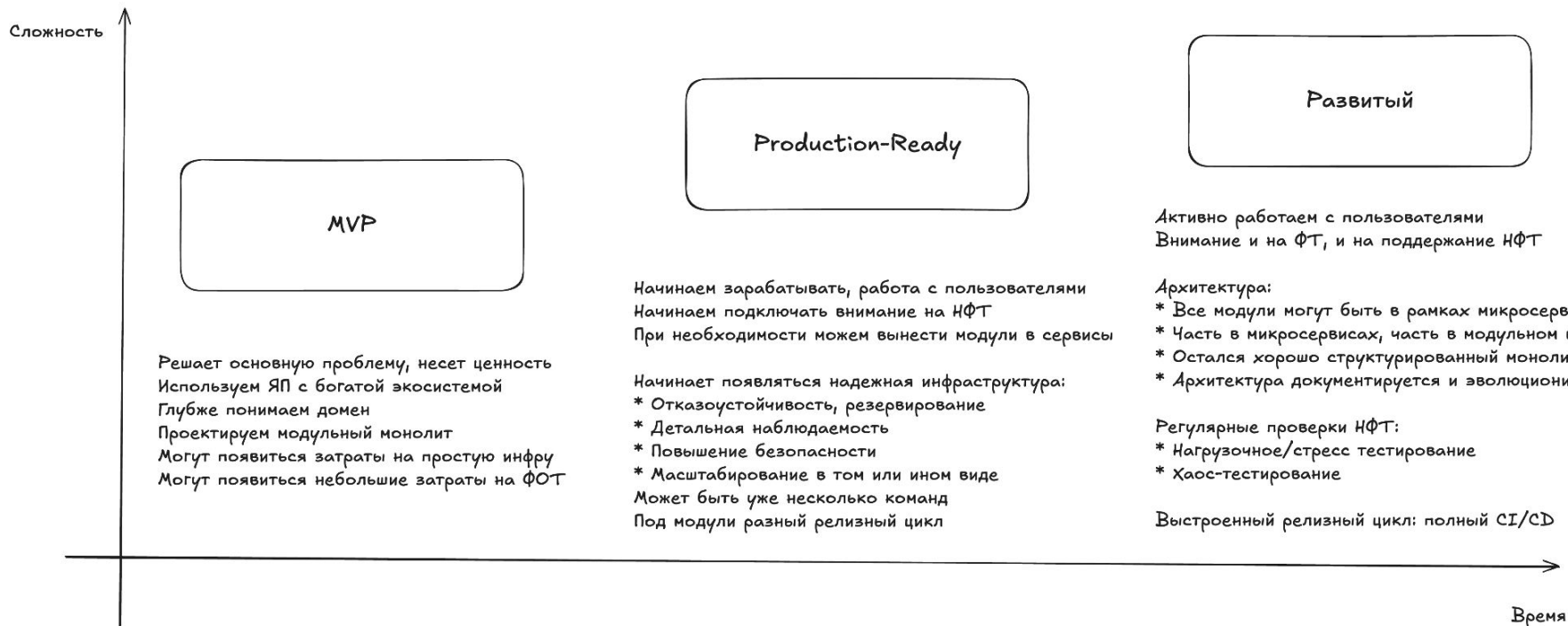
https://systems.education/wis_ddd_event_storming



Фазы продукта [0]



Фазы продукта [1]





Когда можно вынести модуль в МС

Масштабирование

- + Есть потребность делать масштабирование конкретного модуля
- + Локальность масштабирования
- + Проще делать горизонтальное масштабирование stateless МС, чем всего монолита

Деплой

- + Части системы требуют разного деплоя
- + Если модули обновляются одновременно, можем собирать и деплоить одновременно. Но если один раз в месяц, а другой каждый час?

Надежность

- + К разным модулям появляются разные требования к надежности
- + Управлять отказами в МС проще, чем в модульном монолите
- + Работаем с грамотной утилизацией ресурсов при резервировании

Автономность и технологии

- + Нужно повысить автономность команд
- + Два человека не могут мерджить без конфликтов
- + Под разные модули нужны разные языки программирования



Выводы

Архитектура изменчива

На нее могут влиять цели фазы продукта, доступные ресурсы, целесообразность

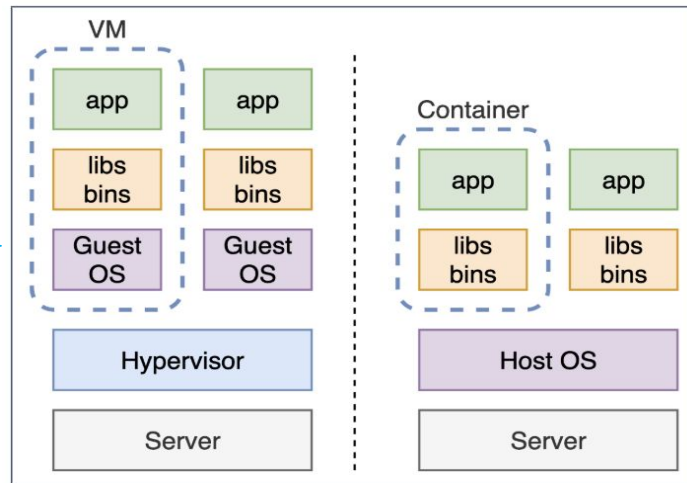
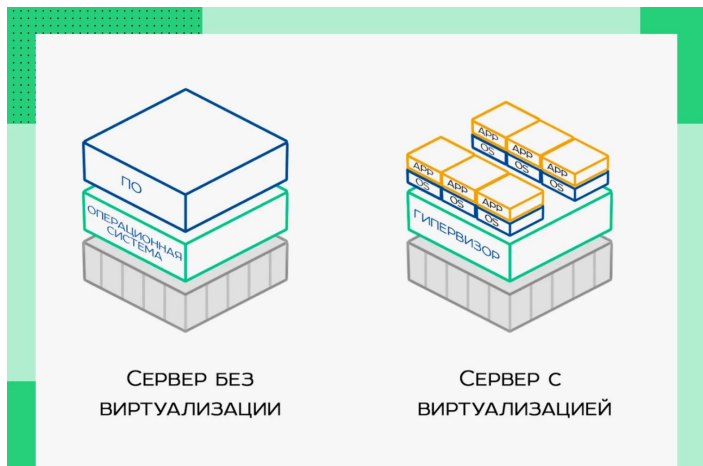
Архитектура обслуживает реальность и эволюционирует в будущее

- Надо проектировать так, как лучше сейчас, но закладывать рычаги на будущее
- Часто встречаются крайности
 - Плохо обслуживаем реальность, потому что заложили на вырост, но обанкротились
 - Плохо эволюционируем в будущее, потому что сделали на коленке и не смогли адаптироваться

Микросервисы не панацея

- На старте неочевиден домен. Некорректно разбив границы в начале дальше заплатим дважды за рефакторинг
- Надо жить по карману. Микросервисы дороже не только по инфраструктуре, но и про инженерное время
 - Монолит: один деплой, один процесс, локальные вызовы, одна транзакция, один лог
 - Микросервисы: N деплоев, service discovery, распределенные транзакции, тестирование сервисов
- Обратный закон Конвея: архитектура может определять команду

Где запускать приложение [0]



Виртуализация

С помощью гипервизора нарезаем физический сервер на несколько VM: гипервизор создает полную копию железа

Каждая VM - отдельная ОС со своим ядром

Контейнеризация

С помощью средств Linux (cgroups, chroot, namespaces) нарезаем изолированные процессы, использующие ядро хостовой операционной системы

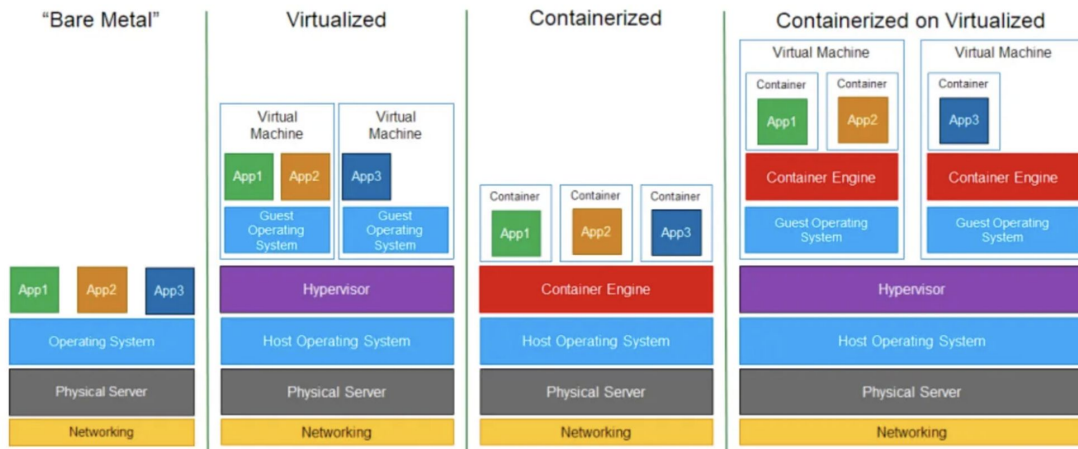
На практике применяются рантаймы (container runtimes), в их числе Docker



Где запускать приложение [1]

Вопрос

Какая из конфигураций наиболее часто встречается в реальности?





Где запускать приложение [2]

Bare Metal

Приложение работает прямо на физическом сервере без прослоек

Что получаем

- + Нет оверхеда от гипервизора (точно)
- + По производительности сопоставимо с контейнерами (<https://blog.hathora.dev/benchmarking-container-performance/>), но нет задержки по сети
- + Контроль над железом, нет шумных соседей

Что теряем

Утилизация ресурсов, гибкость, безопасность

Когда выбираем

Low-latency приложения, большие OLTP-БД, ML-обучение (GPU кластеры), когда требуется строгая изоляция на физическом уровне

Виртуальная машина

Приложение работает, как процесс на гостевой ОС виртуальной машины (напр., VMWare)

Что получаем

- + Изоляция: каждая VM - свое ядро ОС
- + VM можно перенести на другой хост без остановки, состояние VM можно забэкапить и восстановить
- + Лучшую утилизацию железа на сервере

Что теряем

Проблема шумных соседей, дополнительная задержка, дополнительные издержки на ЦПУ и ОЗУ

Когда выбираем

Нужна VM-level изоляция, stateful-приложения, не подходящие под контейнеры

Контейнер

Приложение работает, как изолированный процесс на хостовой ОС (напр., Docker)

Что получаем

- + Практически нет оверхеда относительно Bare Metal по производительности
- + Переносимость, единообразие образов
- + Простота в формировании, обслуживании, масштабировании

Что теряем

Более слабая изоляция относительно VM, сетевые задержки, неэффективность для stateful-приложений

Когда выбираем

Stateless-сервисы (всевозможные веб-серверы, воркеры, API GW), микросервисы, dev-окружения

Оркестрация

Управляет контейнерами, не является способом запуска (напр., k8s)

Что получаем

- + Автоматическое восстановление: упал pod -> автоматически его поднял
- + Автоматическое масштабирование: нагрузка выросла -> автоматически добавил экземпляры
- + Всякие вещи по типу rolling updates, service discovery
- + Ниже time-to-market

Что теряем

Операционная сложность, оверхед от кластера, трудоемкий дебаг, больше мест для ошибки

Когда выбираем

Нестабильная нагрузка, необходимость в быстром масштабировании, частые обновления приложения, есть компетенции, много сервисов

On-Premise / Cloud



ON-PREMISE

Свобода и контроль
ценой
операционной
сложности



CLOUD

Операционная
простота
ценой
зависимости

НФТ	Cloud	On-Premise
Производительность (всплески)	Динамическое масштабирование	Нужно держать резерв по ресурсам
Производительность (стабильная нагрузка)	Риск шумных соседей	Полный контроль
Задержка (внутри системы)	Если МультиЦОД по компонентам, то может быть	Локальная сеть
Масштабируемость	Секунды/минуты	Недели (закупка, настройка серверов)
Надежность (Disaster Recovery)	МультиЦОД, бэкапы из коробки	Если инвестировали
Безопасность (физическая)	В целом, высокая (бывают исключения, см. ближний восток)	Зависит от компании
Безопасность (данные)	Доверяешь провайдеру	Полный контроль
Compliance (финтех, госука)	Нет, если только не Private Cloud	Да
Операционная нагрузка	Провайдер берет на себя	Все на нас
Стоимость (начальная)	Нет Capex	Высокий Capex
Стоимость (высокая нагрузка)	Дорого	Дешевле за каждый юнит
Независимость от вендора	Вендор-лок	Только разве что на оборудование
Скорость инноваций	Доступны managed services	Все самостоятельно



Cloud Native

Что это?

Набор принципов проектирования, при которых приложение умеет использовать возможности облака: горизонтальное масштабирование, самовосстановление, быстрый деплой

Контейнеры

- + Изолированные
- + Переносимые
- + Воспроизводимые

Динамическая оркестрация

- + k8s
- + Сам решает где и сколько контейнеров надо

Микросервисы

- + Независимые
- + Loose coupling

DevOps-культура

- + CI/CD
- + Автоматизация всего доступного
- + Наблюдаемость
- + IaC



Манифест 12-Factor App - методология для создания Cloud Native приложений

1. **Codebase.** Один репозиторий - много окружений (dev/stage/prod)
2. **Dependencies.** Зависимости явно объявлены, прописаны в манифесте
3. **Config.** Код не зависит от окружения, а конфигурация зависит. Поэтому код должен храниться в репозитории, а конфигурация в окружении
4. **Backing services.** БД, очереди, кэши - как заменяемые ресурсы, не должны влиять на код приложения. Приложение не знает где они развернуты
5. **Build/Release/Run.** Сборка, релиз, запуск - строго отдельные этапы
6. **Processes.** Stateless, share-nothing - ничего не храним в памяти между запросами
7. **Port binding.** Сервис сам экспортирует себя через порт
8. **Concurrency.** Масштабирование через процессы, не треды
9. **Disposability.** Быстрый старт, graceful shutdown
10. **Dev/prod parity.** Dev и prod максимально похожи
11. **Logs.** Логи — поток событий, не файлы
12. **Admin processes.** Разовые задачи — как отдельные процессы

Архитектура влияет на инфраструктуру

- Сделали выбор в пользу микросервисов -> нужен k8s, service mesh, трейсинг
- Сделали stateless сервисы -> можно горизонтально масштабировать
- Выбрали асинхронные интеграции -> нужна Kafka или RabbitMQ как часть инфраструктуры
- Разные хранилища -> отдельная инфраструктура и операционная нагрузка под каждую БД
- Репликация, шардирование -> дополнительные косты на инфраструктуру

...



Event-Driven Architecture [0]

Идея

Событийно-ориентированный подход

Разбиваем систему на независимые компоненты, которые общаются между собой через события

Какую проблему решаем

Цепочка синхронных вызовов может оборваться в один из моментов

Что получаем

- Простое масштабирование
- Отказоустойчивость
- Событие - хороший квант информации: неизменяемый факт

Сложности

- Eventual Consistency (согласованность в конечном итоге)
- Порядок событий
- Эволюция схемы
- Тяжелое тестирование

YES,

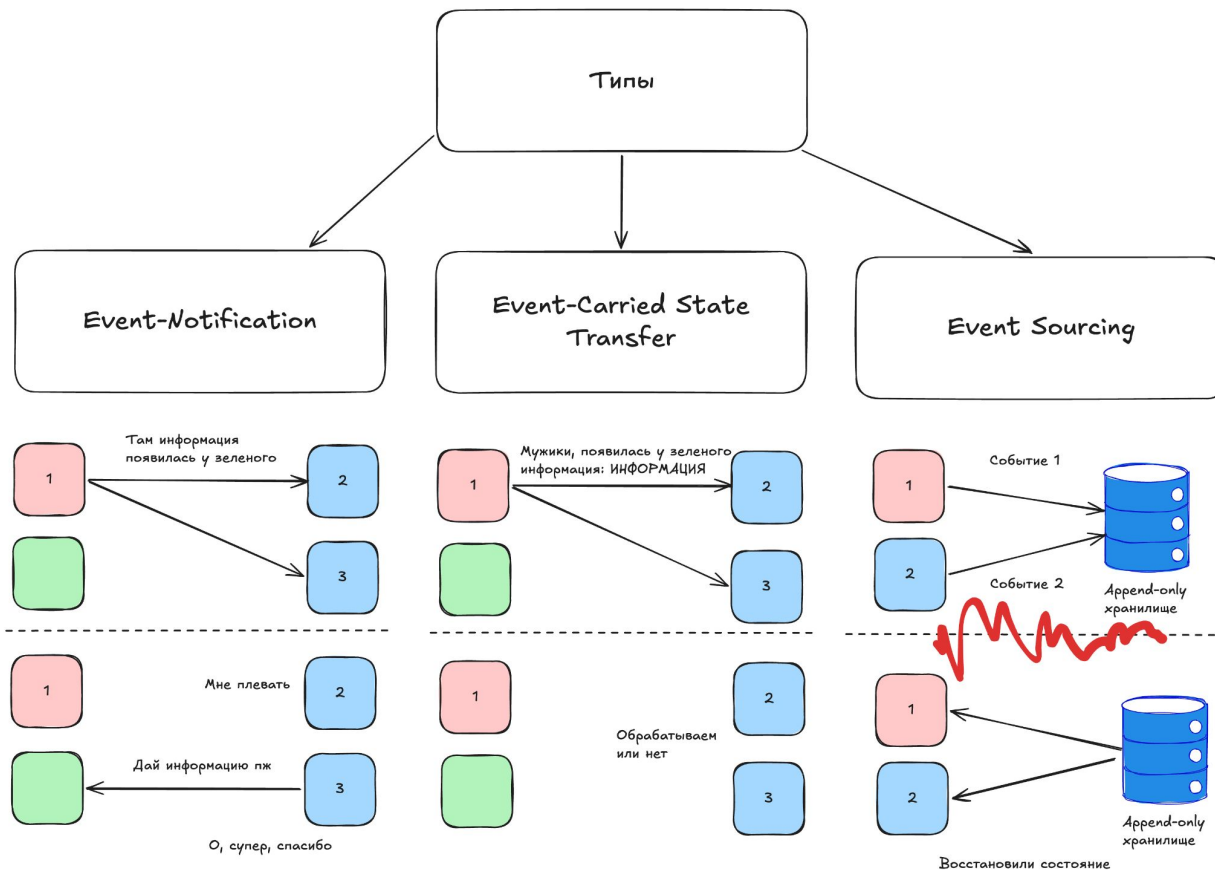
BUT

1. Устойчивость к сбоям
2. Независимое масштабирование
3. Легко добавлять потребителей
4. Лог событий

1. Eventual Consistency
2. Сложность тестирования
3. Версионирование схемы
4. Доп. затраты на транспорт (брокер, очереди)



Event-Driven Architecture [1]





CQRS (Command Query Responsibility Segregation)

Идея

Разделяем операции записи и чтения данных в приложении на две отдельные ветки

Команды могут обрабатываться как в синхронных запросах, так и в асинхронных

Какую проблему решаем

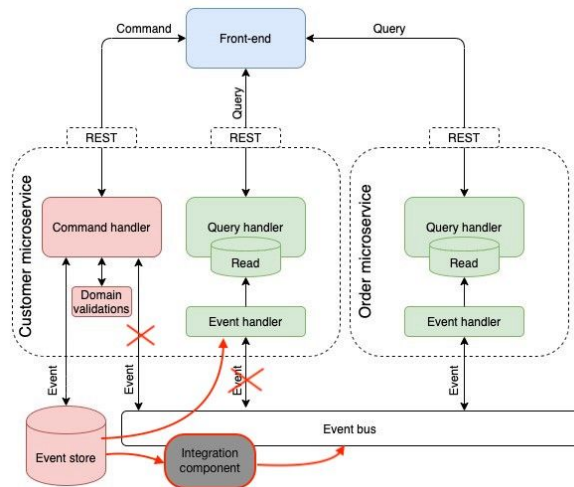
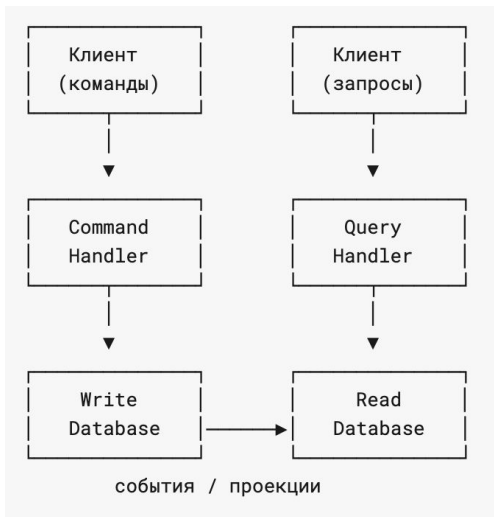
- Разное количество read/write операций (например, e-commerce, где более 10:1)
- Сложные запросы на чтение

Что получаем

- Если много операций записи - справедливое масштабирование
- Улучшаем производительность
- Просто понимать и тестировать

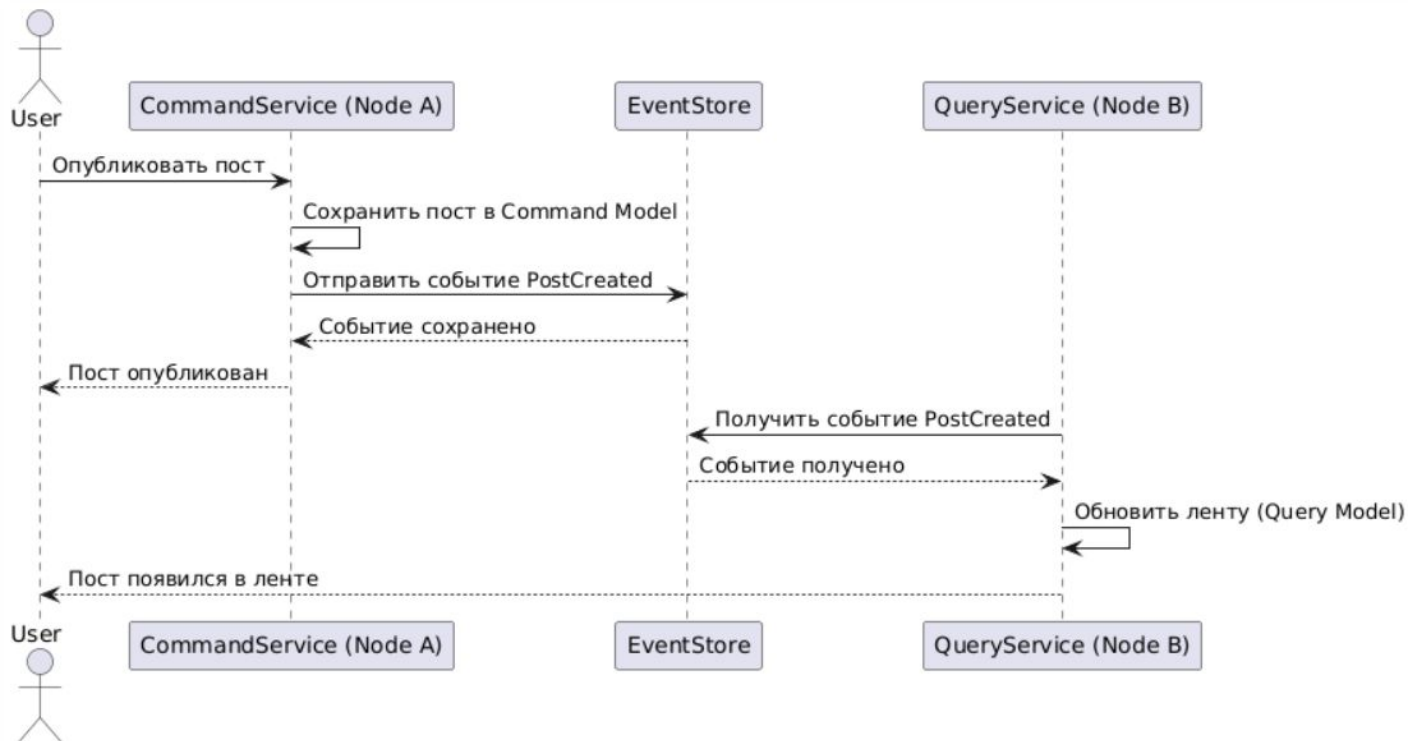
Сложности

- Eventual Consistency (согласованность в конечном итоге)
- Сложность кодовой базы
- Задержка синхронизации





CQRS (Command Query Responsibility Segregation)





Saga (оркестрация и хореография)

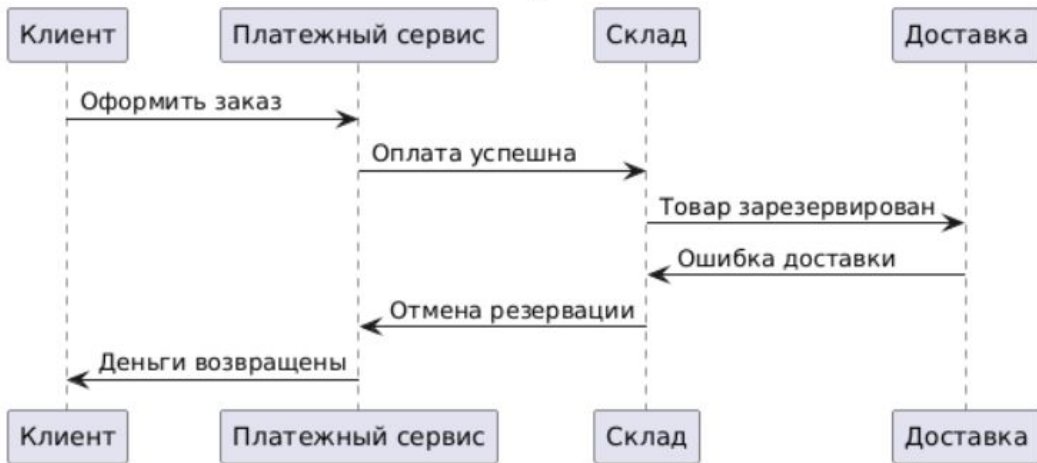
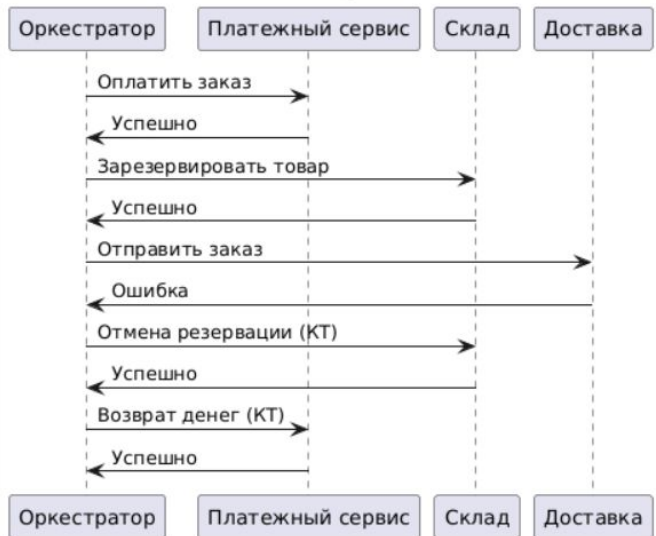
Оркестрация

Какую проблему решаем

Распределенные сервисы, у каждого своя БД

Нужна операция, которая затрагивает несколько сервисов и при ошибке должна откатиться

Хореография



Архитектурные стили и паттерны

- Не призваны заменить монолитную или микросервисную архитектуру, а призваны решить конкретные задачи, заплатив за это конкретную цену - опять компромиссы
- Увеличивают архитектурную сложность, призываю не использовать без надобности, иначе похоже на “используем кубер для строчки в резюме”



Ренессанс модульного монолита

Монолит

У нас монолитное приложение. Вроде вертикально масштабируемся. Сложно стало с разработкой, фичи выпускаем не так часто. Становится неуправляемым...

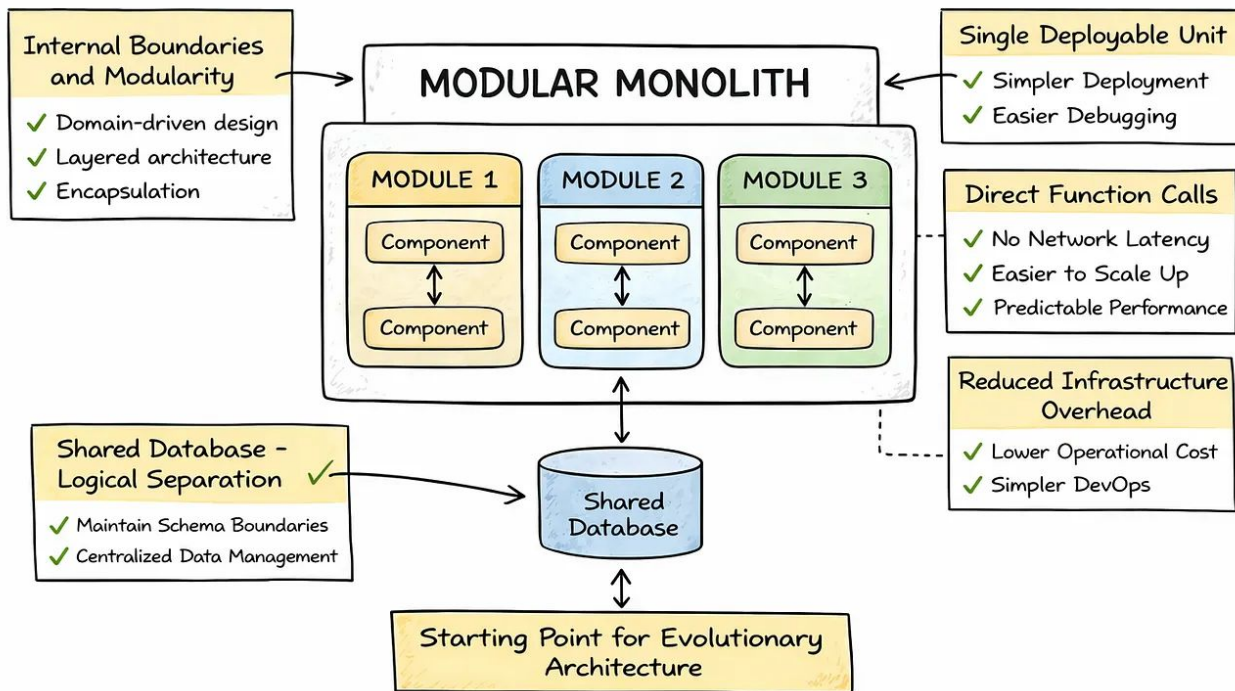
Микросервисы

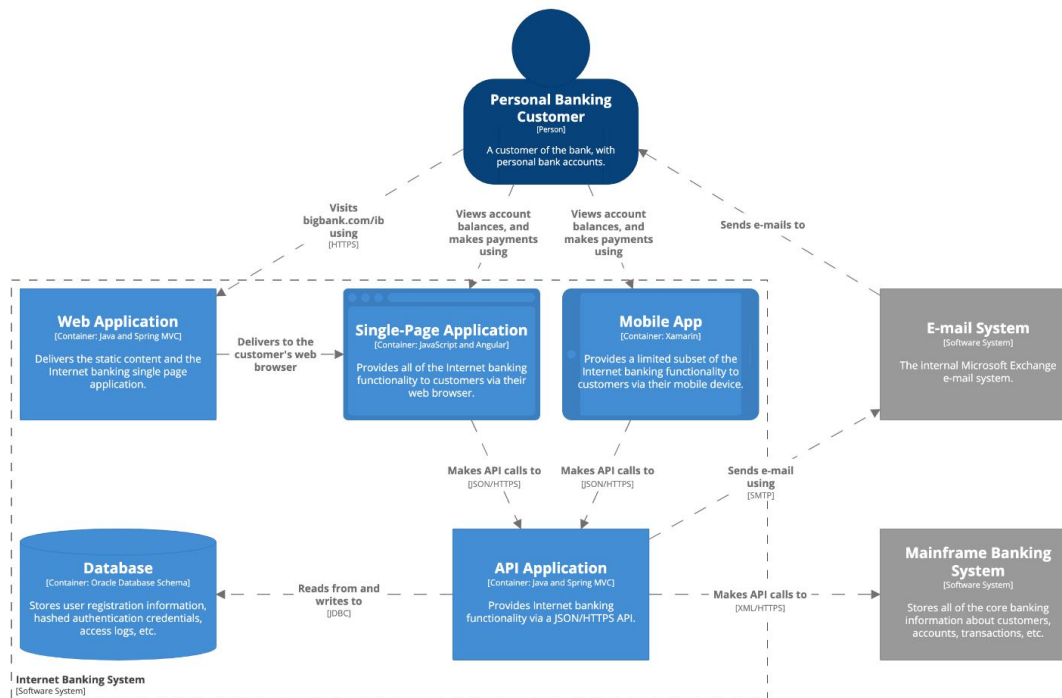
Наш СТО услышал про Docker, какие-то новомодные микросервисы. Мы слышали что-то про SOA, давайте нарежем монолит на микросервисы!

Распределенный монолит

Нарезали...

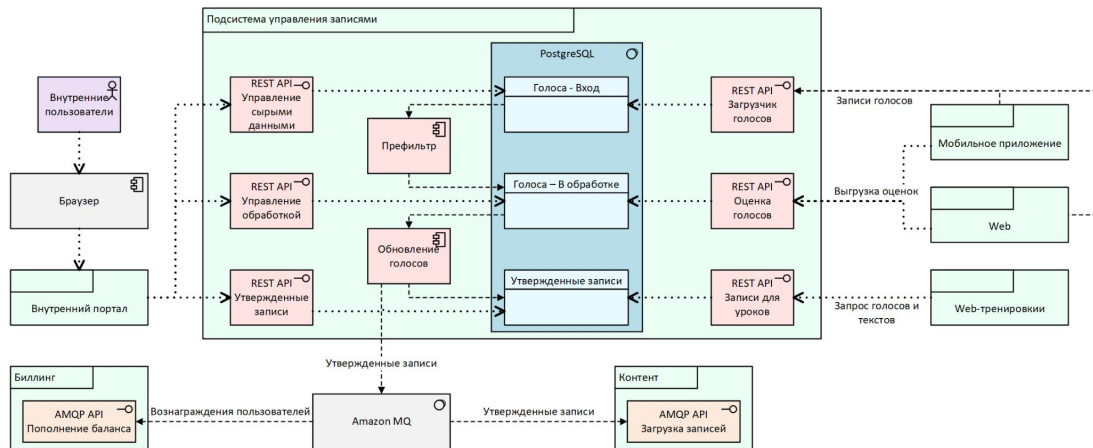
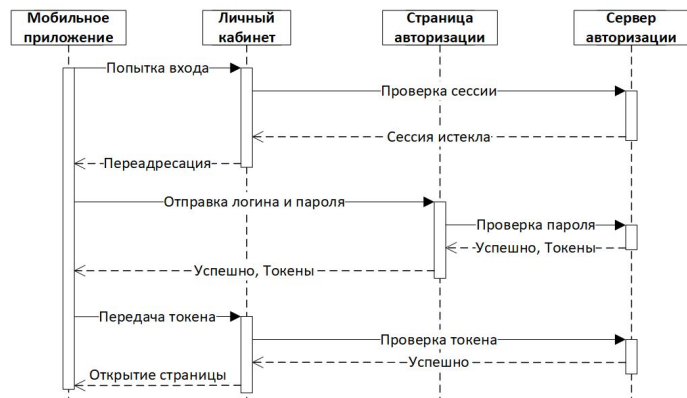
Ренессанс модульного монолита





Container diagram for Internet Banking System

The container diagram for the Internet Banking System.
Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)





Язык программирования больше не важен

Но это не точно

Глобально, с приходом AI в SDLC (Software Development Life Cycle) - мы можем вообще ни разу не увидеть код, как это делают вайбкодеры

Особенности ЯП все равно важны в атрибутах качества системы

Все еще есть операционный опыт: дебажить Prod в 3 ночи, когда закончились токены...

LLM пока неравномерен: Python/TypeScript/Go - отлично знает; Erlang/Nim/Zig - значительно хуже



Just-in-Time архитектура

Если AI быстро пишет код, то можем быстро эволюционировать

Гипотеза: можем меньше принимать стратегических решений, т.к. тактически мы начинаем быстрее адаптироваться под изменение внешних условий, таким образом архитектура может перестраиваться по 3 раза в день

Но это работает для обратимых решений. Все еще важны необратимые: модель данных (миграции болезненны), публичные API контракты (клиенты уже интегрировались), выбор брокера сообщений (миграция данных), модель безопасности приложения



Spec-Driven Development

Раньше: архитектор -> диаграммы -> разработчики -> код

Сейчас: архитектор -> спецификации -> AI -> код -> разработчики проверяют, хотя можно настроить агента-валидатора и уйти в MAS

Спецификации становятся первичным артефактом. OpenAPI, ADR, архитектурные ограничения в машиночитаемом формате - AI это ест и генерирует реализацию

Что это значит для нас: навык писать точные, недвусмысленные спецификации становится важнее навыка писать код



Architecture as docs-as-code

Папка /architecture

/decisions: ADR файлы

/diagrams: C4 модели в коде (Structurizr DSL, Mermaid)

/constraints: ArchUnit правила

/specs: OpenAPI, AsyncAPI

CI/CD проверяет:

ADR существует для каждого значимого решения

диаграммы актуальны (сгенерированы из кода)

ArchUnit не нарушен



LLM знает только open source

Обучен на публичном коде (если не брать дообучение)

Знает хорошо:

- PostgreSQL, Kafka, Redis, Kubernetes, React, ...
- стандартные паттерны

Не знает:

- внутренние фреймворки компании
- проприетарные платформы
- легаси-системы без документации в интернете

Аргумент против самописных фреймворков и за стандартные решения



Самолечение архитектуры

Архитектурный уровень

AI-агент мониторит метрики архитектуры (связанность, зависимости, деградация производительности), обнаруживает сдвиг, предлагает рефакторинг или даже применяет его автоматически в пределах заданных границ



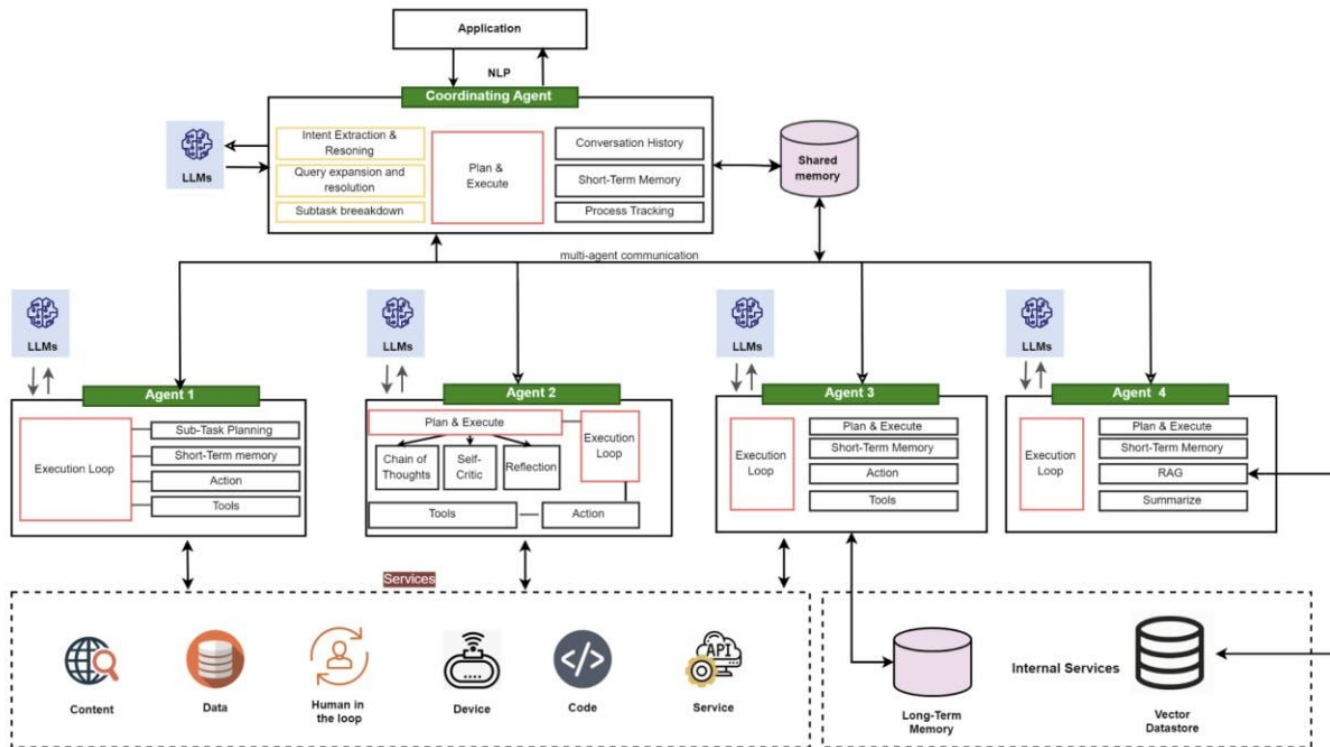
Все ресурсы у ИИ - что делать?

Стоит задуматься

Возможные тренды:

- Маленькие модели и edge-серверы: вычисления ближе к пользователю
- AI как инфраструктура: вместо "арендуй ВМ, разворачивай приложение" - "опиши задачу AI-агенту, он сделает"
- Аппаратная экономия, конкуренция корпоративных систем за ресурсы, максимизация утилизации

AI AGENTIC ARCHITECTURE





НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ